Django Readiness Challenge Set (Real-World Style)

These exercises are designed to test whether a learner is truly ready to begin Django development. Each project combines Python fundamentals, OOP, SQL thinking, debugging, data relationships, HTTP concepts, validation, and backend logic in a real-world style scenario. HTML-related tasks have intentionally been excluded.

1. School Management System (Console-Based)

Scenario: A school wants a system to manage students, teachers, courses, and enrollments.

- Register students with full name, age, email, and department.
- Add teachers and assign courses to them.
- Create courses with course code, title, and unit.
- Allow students to enroll in multiple courses.
- Store records in files so data persists after restart.
- Prevent duplicate emails and invalid inputs.
- Allow searching by student ID, student name, and course code.
- Write SQL queries for inserting students, retrieving enrollments, updating courses, and deleting teachers.
- Explain whether registration should use GET or POST requests and why.
- Handle file errors, invalid input, and index errors.

2. Hospital Patient Record System

Scenario: A clinic wants software to manage patients, doctors, appointments, and billing.

- Register patients with patient ID, full name, age, gender, and phone number.
- Allow patients to choose doctors and appointment dates.
- Prevent duplicate appointment bookings.
- Calculate consultation fees, medication fees, and total bills.
- Use inheritance with Person, Patient, and Doctor classes.
- Explain One-to-One, One-to-Many, and Many-to-Many relationships using hospital examples.
- Save patient records into files.
- Handle invalid dates, missing records, and wrong age inputs.
- Write SQL queries for searching patients, retrieving appointments, calculating bills, and updating records.
- Explain the complete backend flow from request to database response.

3. E-Commerce Inventory & Order System

Scenario: Build a mini shopping system for an online store.

- Allow admins to add, update, and remove products.
- Allow customers to register, login, and place orders.
- Implement cart logic for multiple products and automatic totals.
- Use functions, loops, conditionals, and dictionaries.
- Create User, Customer, Admin, Product, and Order classes.
- Validate usernames and passwords during login.
- Write SQL queries for product insertion, stock updates, retrieving orders, and deleting products.
- Explain sessions and why login requests should use POST.
- Handle invalid quantities, empty carts, duplicate products, and failed logins.
- Explain how to prevent negative stock and duplicate usernames.

4. Banking/ATM System

Scenario: Create a banking system supporting account creation, login, deposits, withdrawals, and transfers.

- Store account number, full name, email, PIN, and balance.
- Allow deposits, withdrawals, and money transfers.
- Store transaction history.
- Prevent overdrafts, invalid transfers, and duplicate accounts.
- Use encapsulation, constructors, and methods properly.
- Persist account data after restart using files.
- Handle invalid PINs and missing accounts.
- Write SQL queries for creating accounts, updating balances, retrieving transactions, and transferring money safely.
- Explain database transactions and what happens if transfers fail halfway.
- Explain why banking operations should use POST requests.

5. Student Result Processing Portal

Scenario: A school wants a result portal for lecturers and students.

- Allow lecturers to upload, edit, and delete scores.
- Allow students to login and view results.
- Automatically calculate GPA using course units and grade points.
- Use loops, functions, dictionaries, and conditionals.
- Create Student, Lecturer, Course, and Result classes.
- Model relationships between students, lecturers, and courses.
- Write SQL queries for retrieving results, GPA calculations, updating scores, and deleting records.
- Prevent scores above 100 and duplicate course entries.
- Explain authentication flow and session handling.

- Handle invalid grades and missing student records.

Evaluation Standard

- Can structure medium-to-large programs logically.
- Understands OOP concepts and data relationships.
- Can debug independently.
- Understands SQL and CRUD operations.
- Understands request/response flow.
- Can validate and persist data properly.
- Can reason about backend systems instead of memorizing syntax.